

DSA

Module 7

Strings

Author
Srinivas Dande





Java Learning Center

No.1 In Java Training & placement



7. Strings

- ◆ String is a Sequence of Characters.
- ◆ String can be represented in java in the following ways
 - 1) String class
 - 2) StringBuffer
 - 3) StringBuilder
 - 4) Array of Characters
 - 5) ArrayList of Characters

7.1. String Class

- ◆ String is a built-in class in java.lang package.
- ◆ String is final class, so you can't define the subclass for String class.
- ◆ String class implements the following interfaces:
 - ◆ java.io.Serializable
 - ◆ java.lang.Comparable
 - ◆ java.lang.CharSequence
- ◆ String class has following variable to hold data.

```
private final char value[];
```
- ◆ String objects are immutable objects. It means once the object is created then the content or data of the object can't be modified.
- ◆ When you try to modify the contents of object then new object will be created as a result.
- ◆ You can create the object of String in two ways.
 - ◆ Without using new operator.
 - ◆ Using new operator.

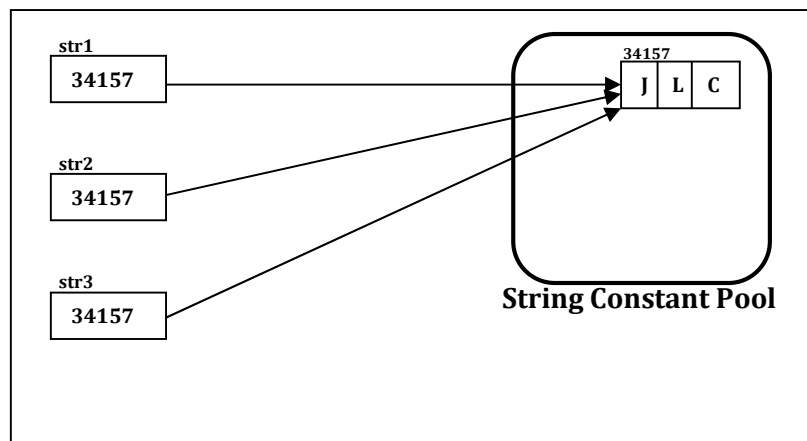


7.1.1. Creating String object without new operator

- 1) JVM allocates the memory for the String reference variable.
- 2) JVM verifies the String literal in the String Constant Pool.
- 3) If String literal is not available in the String Constant Pool then it creates new String object inside the String Constant pool and newly created String object address will be assigned to String reference variable.
- 4) If String literal is available then existing String object address will be assigned to String reference variable.

Demo1.java

```
class Lab457{  
    public static void main(String as[]){  
        String str1="JLC";  
        String str2="JLC";  
        String str3="JLC";  
        System.out.println(str1==str2);  
        System.out.println(str1==str3);  
        System.out.println(str2==str3);  
    }  
}
```

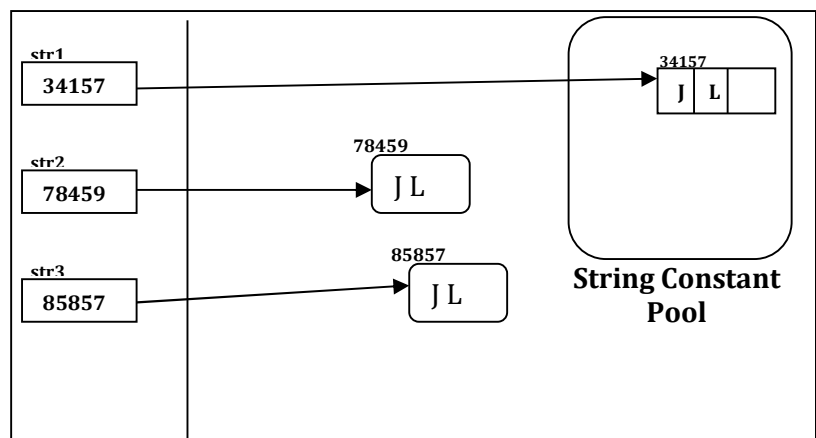


7.1.2 Creating String object with new operator

- 1) JVM allocates the memory for the String reference variable.
- 2) JVM verifies the String literal in the String Constant Pool.
- 3) If String literal is not available in the String Constant Pool then it creates new String object inside the String Constant pool.
- 4) If String literal is available in the String Constant Pool then ignores that.
- 5) It creates another new String object outside the constant pool and assigns address of the newly created String object outside the pool to String reference variable.

Demo2.java

```
class Lab458{  
    public static void main(..... as[]){  
        String str1="JLC";  
        String str2=new String("JLC");  
        String str3=new String("JLC");  
        System.out.println(str1==str2);  
        System.out.println(str1==str3);  
        System.out.println(str2==str3);  
    }  
}
```



7.2. StringBuffer Class

- ◆ StringBuffer is a final class in java.lang package.
- ◆ It is used to store the sequence of characters.
- ◆ This is used when there is a necessity to make a lot of modifications to characters of String.
- ◆ StringBuffer is a mutable sequence of characters, It means the contents of the StringBuffer can be modified after creation.
- ◆ Every StringBuffer object has a capacity associated with it.
- ◆ The capacity of the StringBuffer is the number of characters it can hold.
- ◆ The capacity increases automatically as more contents added to it.
- ◆ The methods available in the StringBuffer class are synchronized.
- ◆ StringBuffer class is thread-safe i.e. multiple threads cannot access it simultaneously.

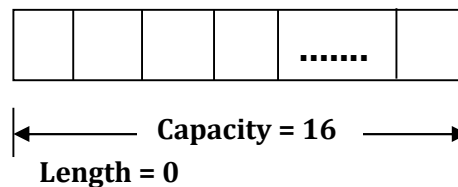
7.3. StringBuilder Class

- ◆ StringBuilder is a final class in java.lang package.
- ◆ StringBuilder is a newly added class from Java 5.
- ◆ StringBuilder class functionality is same as StringBuffer only except the methods available in the StringBuilder class are non synchronized.
- ◆ StringBuilder class is not thread-safe i.e. multiple threads can access it simultaneously.

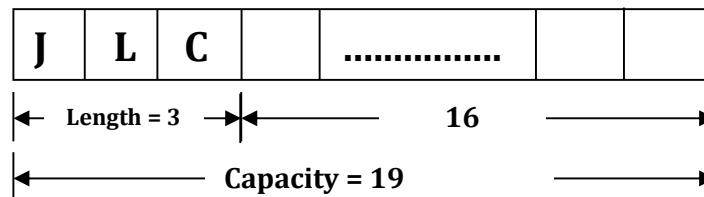
7.4. About StringBuffer and StringBuilder

- ♦ equals() is not overridden in StringBuilder and StringBuffer class.
- ♦ When you want to compare contents of StringBuilder or StringBuffer object then you need to convert the object of StringBuilder or StringBuffer into String.
- ♦ hashCode() is not overridden in the StringBuilder and StringBuffer class. It uses the default implementation of Object class.
- ♦ StringBuilder and StringBuffer are not implementing java.lang.Comparable interface, So You can't use compareTo() with StringBuilder and StringBuffer.

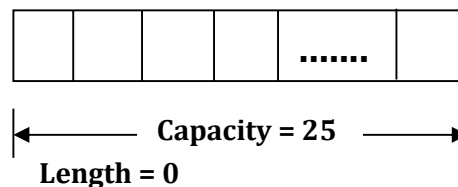
StringBuilder sb=new StringBuilder();



StringBuilder sb=new StringBuilder("JLC");



StringBuilder sb=new StringBuilder(25);



7.5. Examples on Strings

Lab1.java

```
package com.jlcindia.strings;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab1 {
    public static void main(String[] args) {

        String str1="JLC";
        String str2="JLC";

        System.out.println(str1==str2);

        String str3=new String("JLC");
        System.out.println(str1==str3);

    }
}
```



Lab2.java

```
package com.jlcindia.strings;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab2 {
    public static void main(String[] args) {

        String str1="Hello";
        String str2=" Guys";

        String str3="Hello Guys";
        String str4= str1.concat(str2);
        String str5="Hello".concat(" Guys");

        System.out.println(str3==str4); //F
        System.out.println(str3==str5); //F
        System.out.println(str4==str5); //F

        String str6=str1+str2;
        String str7="Hello" + " Guys"; //IMP- Literal + Literal

        System.out.println(str3==str6);
        System.out.println(str3==str7);

    }
}
```



Lab3.java

```
package com.jlcindia.strings;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab3 {
    public static void main(String[] args) {

        String str1="JLCD";
        String str2="JLCD";
        String str3="JLCd";

        System.out.println(str1.equals(str2));

        System.out.println(str3.equals(str2));
        System.out.println(str3.equalsIgnoreCase(str2));

        System.out.println(str1.compareTo(str2)); //Same
        System.out.println(str2.compareTo(str3)); //str2 is smaller
        System.out.println(str3.compareTo(str2)); //str3 is Bigger

    }
}
```



Lab4.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab4 {
    public static void main(String[] args) {

        String str="srinivasd";

        System.out.println(str);
        System.out.println(str.length());

        char chArr[] = str.toCharArray();

        for(int i=0;i<chArr.length;i++)
            System.out.print(chArr[i]+" ");

        System.out.println("\n");

        chArr[0]='S';
        chArr[8]='D';

        for(int i=0;i<chArr.length;i++)
            System.out.print(chArr[i]+" ");

        System.out.println("\n");

        String str1=new String(chArr);
        System.out.println(str1);
        System.out.println(str);
    }
}
```



Lab5.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab5 {
    public static void main(String[] args) {

        String str="srinivasd";

        System.out.println(str.length());
        System.out.println(str.charAt(0));
        System.out.println(str.charAt(8));

        System.out.println(str.substring(5));
        System.out.println(str.substring(3,8));
        System.out.println(str.substring(3,9));

        System.out.println(str.indexOf("nivas"));
        System.out.println(str.indexOf("i"));
        System.out.println(str.lastIndexOf("i"));

        System.out.println(str.contains("vas"));
        System.out.println(str.contains("nivas"));
        System.out.println(str.contains("hello"));

    }
}
```



Lab6.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab6 {
    public static void main(String[] args) {

        StringBuffer sb1=new StringBuffer("JLC");
        StringBuffer sb2=new StringBuffer("JLC");

        System.out.println(sb1==sb2);

        System.out.println(sb1.equals(sb2));

        boolean b1= sb1.toString().contentEquals(sb2.toString());
        boolean b2= sb1.toString().equals(sb2.toString());

        System.out.println("b1 : "+b1);
        System.out.println("b2 : "+b2);

        sb1.append(" Guys");
        System.out.println(sb1);

    }
}
```



Lab7.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab7 {
    public static void main(String[] args) {

        StringBuilder sb1=new StringBuilder("JLC");
        StringBuilder sb2=new StringBuilder("JLC");

        System.out.println(sb1==sb2);

        System.out.println(sb1.equals(sb2));

        boolean b1= sb1.toString().contentEquals(sb2.toString());
        boolean b2= sb1.toString().equals(sb2.toString());

        System.out.println("b1 : "+b1);
        System.out.println("b2 : "+b2);

        sb1.append(" Guys");
        System.out.println(sb1);

        sb1.reverse();
        System.out.println(sb1);
    }
}
```



7.6. Problems on Strings

Problem1A.java

```
package com.jlcindia.strings;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Problem1- Reverse the given String

public class Problem1A {
    public static void main(String[] args) {

        String str = "Hello";
        System.out.println(str);

        StringBuffer sb=new StringBuffer(str);

        sb.reverse();

        String revStr = sb.toString();
        System.out.println(revStr);

    }
}

// Time Complexity - - O(1)
// Aux Space - O(1)
```



Problem1B.java

```
package com.jlccindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem1- Reverse the given String
public class Problem1B {
    static String reverseString(String str) {

        char chArr[] = str.toCharArray();
        int n = chArr.length;
        int start=0;
        int end=n-1;
        while(start<end) {
            char temp = chArr[start];
            chArr[start] = chArr[end];
            chArr[end]=temp;

            start++;
            end--;
        }
        String revStr=new String(chArr);
        return revStr;
    }
    public static void main(String[] args) {
        String str = "Hello";
        System.out.println(str);

        String revStr = reverseString(str);
        System.out.println(revStr);
    }
}

// Time Complexity - - O(n/2)/O(n)
// Aux Space - O(n)
```



Problem2.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem2- Reverse Words of given Sentence

public class Problem2 {

    static void reverse(char chArr[], int start, int end) {
        while (start < end) {
            char temp = chArr[start];
            chArr[start] = chArr[end];
            chArr[end] = temp;

            start++;
            end--;
        }
    }

    static void reverseWords(char chArr[]) {
        int n = chArr.length;
        int start = 0;
        int end = 0;
        while (end < n) {
            if (chArr[end] == ' ') {
                reverse(chArr, start, end - 1);
                start = end + 1;
            }
            end++;
        }
        reverse(chArr, start, end-1); // Reverse Last Word
        reverse(chArr, 0, n-1); //Reverse the Entire String
    }
}
```



```
public static void main(String[] args) {  
  
    String str = "SD OK";  
    System.out.println(str);  
  
    char chArr[] = str.toCharArray();  
    reverseWords(chArr);  
  
    String revStr= new String(chArr);  
    System.out.println(revStr);  
  
    }  
}
```

//Time Complexity - $O(3n)$ / $O(n)$

//Aux Space --- $O(n)$



Problem3.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem3- Frequency of each Character of given String with lower
// Alphabets in Sorted Order
public class Problem3 {

    static void charFrequency(String str) {
        int n= str.length();

        int count[] = new int[26];

        for(int i=0;i<n;i++) {
            char ch=str.charAt(i);
            count[ch-97]++;
        }

        for(int i=0;i<count.length;i++) {
            if(count[i]>0) {
                char ch = (char) (97+i);
                System.out.println(ch + "\t"+count[i]);
            }
        }
    }

    public static void main(String[] args) {
        String str = "srinivas";
        charFrequency(str);
    }
}

//Time Complexity - O(n)
//Aux Space --- O(26)/O(1)
```



Problem4A.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P4- Valid Palindrome
//Check whether given string is Palindrome or Not.

public class Problem4A {

    static boolean isPalindrome(String str) {

        StringBuilder sb=new StringBuilder(str); //O(1)
        sb.reverse(); // O(n)
        String revStr=sb.toString(); //O(1)

        return str.equals(revStr);// O(n)
    }

    public static void main(String[] args) {

        String str1 = "abcba";
        boolean b1 = isPalindrome(str1);
        System.out.println(b1);

        String str2 = "abc";
        boolean b2 = isPalindrome(str2);
        System.out.println(b2);

    }
}

//Time Complexity - O(n)
//Aux Space Complexity - O(n)
```



Problem4B.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P4- Valid Palindrome
//Check whether given string is Palindrome or Not.

public class Problem4B {
    static boolean isPalindrome(String str) {
        int n = str.length();
        int start = 0;
        int end = n - 1;

        while (start < end) {
            if (str.charAt(start) != str.charAt(end)) {
                return false;
            }

            start++;
            end--;
        }
        return true;
    }
    public static void main(String[] args) {

        String str1 = "abccba";
        boolean b1 = isPalindrome(str1);
        System.out.println(b1);
    }
}

//Time Complexity -  $O(n/2) \Rightarrow O(n)$ 
//Aux Space Complexity -  $O(1)$ 
```



Problem5A.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
import java.util.Arrays;

//P5- Valid Anagrams
//Check whether Two strings are anagrams are or not
public class Problem5A {
    static boolean isAnagram(String str1,String str2) {

        if(str1.length()!=str2.length())
            return false;

        char charr1[] = str1.toCharArray();
        Arrays.sort(charr1);
        str1 = new String(charr1);

        char charr2[] = str2.toCharArray();
        Arrays.sort(charr2);
        str2 = new String(charr2);

        return str1.equals(str2);
    }
    public static void main(String[] args) {
        String str1 = "sir";
        String str2 = "sri";

        boolean b1 = isAnagram(str1,str2);
        System.out.println(b1);
    }
}

//Time Complexity - O(n log(n))
//Aux Space Complexity - O(m+n) // O(n)
```



Problem5B.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P5- Valid Anagrams
//Check whether Two strings are anagrams are or not
public class Problem5B {
    static boolean isAnagram(String str1,String str2) {
        if(str1.length()!=str2.length())
            return false;

        int counter[] = new int[26];

        for(int i=0;i<str1.length();i++) {
            counter[str1.charAt(i) - 97]++;
            counter[str2.charAt(i) - 'a']--;
        }

        for(int i=0;i<counter.length;i++) {
            if(counter[i]!=0) {
                return false;
            }
        }
        return true;
    }
    public static void main(String[] args) {
        String str1 = "abc";
        String str2 = "cba";
        boolean b1 = isAnagram(str1,str2);
        System.out.println(b1);
    }
}
//Time Complexity - O(n+26) / O(n)
//Aux Space Complexity - O(26) // O(1)
```



Problem6A.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P6- First Repeating Character in the given String
//Consider the String with Lower Alphabets

public class Problem6A {

    static int firstRepeat(String str) {
        int n = str.length();
        for(int i=0;i<n;i++) {
            for(int j=i+1;j<n;j++) {
                if(str.charAt(i) == str.charAt(j)) {
                    return i;
                }
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        String str = "abcdcdef";
        int index = firstRepeat(str);
        System.out.println(index);
    }
}

//Time Complexity - O(n^2)
//Aux Space Complexity - O(1)
```



Problem6B.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P6- First Repeating Character in the given String
//Consider the String with Lower Alphabets

public class Problem6B {

    static int firstRepeat(String str) {
        int counter[] = new int[26];

        for(int i=0;i<str.length();i++) {
            counter[str.charAt(i) - 'a']++;
        }

        for(int i=0;i<str.length();i++) {
            if(counter[str.charAt(i)-97]>1) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        String str = "abcdcdef";
        int index = firstRepeat(str);
        System.out.println(index);
    }
}

//Time Complexity -  $O(2n) \Rightarrow O(n)$ 
//Aux Space Complexity -  $O(26) \Rightarrow O(1)$ 
```



Problem7A.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P7- First Unique Character in the given String
//Consider the String with Lower Alphabets

public class Problem7{

    static int firstUnique(String str) {

        int counter[] = new int[26];

        for(int i=0;i<str.length();i++) {
            counter[str.charAt(i) - 'a']++;
        }

        for(int i=0;i<str.length();i++) {
            if(counter[str.charAt(i)-97]==1) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        String str = "abcdabc";
        int index = firstUnique(str);
        System.out.println(index);
    }
}

//Time Complexity - O(2n) => O(n)
//Aux Space Complexity - O(26) => O(1)
```



Problem7B.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P7- First Unique Character in the given String
//Consider the String with AlphaNumeric (A-Z,a-z,0-9):

public class Problem7B{

    static int firstUnique(String str) {

        int counter[] = new int[75];

        for(int i=0;i<str.length();i++) {
            counter[str.charAt(i) - '0']++;
        }

        for(int i=0;i<str.length();i++) {
            if(counter[str.charAt(i)-48]==1) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        String str = "1Ab2D1cd5Ab";
        int index = firstUnique(str);
        System.out.println(index);
    }
}
//Time Complexity - O(2n) => O(n)
//Aux Space Complexity - O(75) => O(1)
```



Problem7C.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P7- First Non-Repeating Character in the given String
//Consider the String with Alphabets (A-Z,a-z):

public class Problem7C{

    static int firstUnique(String str) {

        int counter[] = new int[58];

        for(int i=0;i<str.length();i++) {
            counter[str.charAt(i) - 'A']++;
        }

        for(int i=0;i<str.length();i++) {
            if(counter[str.charAt(i)-65]==1) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        String str = "AbDcdAbD";
        int index = firstUnique(str);
        System.out.println(index);
    }
}
//Time Complexity - O(2n) => O(n)
//Aux Space Complexity - O(58) => O(1)
```



Problem7D.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P7- First Non-Repeating Character in the given String
//Consider the String with Special Symbols, Alphanumerics

public class Problem7D{

    static int firstUnique(String str) {

        int counter[] = new int[95];

        for(int i=0;i<str.length();i++) {
            counter[str.charAt(i) - ' ' ]++;
        }

        for(int i=0;i<str.length();i++) {
            if(counter[str.charAt(i)-32]==1) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        String str = "#A1*2$BcdF#A1    *2BcdF#A1*2BcdF#A1*2BcdF";
        int index = firstUnique(str);
        System.out.println(index);
    }
}

//Time Complexity - O(2n) => O(n)
//Aux Space Complexity - O(95) => O(1)
```



Problem8.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P8- IsSubSequence

public class Problem8 {
    static boolean isSubsequence(String main,String sub) {
        int m= main.length();
        int n= sub.length();

        if(m<n)
            return false;

        int i= 0;
        int j=0;

        while(i<m && j<n) {
            if(main.charAt(i)==sub.charAt(j)) {
                j++;
            }
            i++;
        }
        return (j==n);
    }
    public static void main(String[] args) {
        String main = "srinivas";
        String sub = "snvs";
        boolean b= isSubsequence(main,sub);
        System.out.println(b);
    }
}
//Time Complexity - O(n)
// Space Complexity - O(1)
```



Problem9.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//P9- Remove Duplicate Characters
public class Problem9 {
    static String removeDuplicates(String str) {

        int n= str.length();
        int count[] = new int[26];

        for(int i=0;i<n;i++) {
            count[str.charAt(i)-97]++;
        }

        StringBuffer sb= new StringBuffer();
        for(int i=0;i<count.length;i++) {
            if(count[i]!=0) {
                char ch= (char)(i+97);
                sb.append(ch);
            }
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        String str = "bacacdbac";
        String mystr= removeDuplicates(str);
        System.out.println(mystr);
    }
}

//Time Complexity => O(n)
// Space Complexity => O(1)
```



Problem10.java

```
package com.jlcindia.strings;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//P10- Reverse the Vowels of String

public class Problem10 {

    public boolean isVowel(char ch) {
        boolean b = ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U';
        return b;
    }

    public String reverseVowels(String str) {
        char chArr[] = str.toCharArray();

        int n= chArr.length;
        int start = 0;
        int end = n-1;

        while(start<end) {

            while(start<n && !isVowel(chArr[start])) {
                start++;
            }

            while(end>=0 && !isVowel(chArr[end])) {
                end--;
            }


```



```
        if(start<end) {
            char temp = chArr[start];
            chArr[start] = chArr[end];
            chArr[end] = temp;
        }

        start++;
        end--;

    }

    String resStr= new String(chArr);
    return resStr;
}

public static void main(String[] args) {

    String str = "srinivaas";
    String mystr = reverseVowels(str);
    System.out.println(mystr);

}
}
```

//Time Complexity - O(n)
// Space Complexity - O(n)